0078. Subsets

y: Medium | Category: Backtracking | LeetCode: https://leetcode.com/problems/s

Problem

- Return all subsets (the power set) of an array of distinct integers.

Intuition

- Each element is either in or out: a binary decision tree of depth n with 2^n leaves.
- Backtracking variant: at each call, record the current subset, then try adding each later element.

Brute Force

- Bitmask enumeration 0..2^n-1 is equally optimal and a nice alternative to show.

Optimal Approach

- Dfs(start): record a copy of path.
- For i = start..n-1: add nums[i], dfs(i+1), remove.

Worked Example

```
[1,2,3].

[] -> [1] -> [1,2] -> [1,2,3]
          -> [1,3]
   -> [2] -> [2,3]
   -> [3]
8 subsets
```

Java Solution

```java
public class Subsets {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        dfs(nums, 0, new ArrayList<>(), res);
        return res;
    }

    private void dfs(int[] nums, int start, List<Integer> path, List<List<Integer>> res) {
        res.add(new ArrayList<>(path));
        for (int i = start; i < nums.length; i++) {
            path.add(nums[i]);
            dfs(nums, i + 1, path, res);
            path.remove(path.size() - 1);
        }
    }
}
```

Complexity

- Time: O(n 2^n). Space: O(n) recursion (excluding output).

Pitfalls

- Add a copy (new ArrayList<>(path)), not the shared list.